

Our database 12 collections

One collection per CSV file. Relationships are resolved at query time with a lookup, not by foreign keys.

DATA COLLECTIONS (one row = one document)

demand

generation

transmission

connections

adequacy_actual

balancing_market

balancing_market_input

intraday_market

day_ahead_market

adequacy_forecast

REFERENCE COLLECTIONS (linked via lookup)

zone

is_italian flag
linked by: demand, transmission,
balancing, intraday, day_ahead

source

renewable flag
linked by: generation

Same shape as SQL, one difference:

no constraints are enforced. The primary key is the automatic `_id` (ObjectId), and links are done only when a query needs them.

We initially considered Oracle Spatial for Query 4, since it handles geographic data. However, it required exact coordinates for each zone, which Terna's dataset does not include and which we could not reliably attribute to the Italian regions and foreign countries involved. Neo4j let us model the same query as a graph of connected zones, without needing those coordinates

Why these systems, and how we set it up

Why MongoDB

- Each row is a flat document: keys and values; one file becomes one collection.
- Schemaless: loading 12 different files was immediate.
- Native Date type, so day-based queries work directly.

Why also Neo4j (Query 4)

- Query 4 is about pairs of connected zones: that is naturally a graph.
- Zones become nodes, each margin becomes an arrow between two zones.
- A join + group-by becomes a network we can read on a map.

All 10 SQL queries reproduced in MongoDB same meaning, verified same results

Three things about the setup

1

Field names

A dot isn't allowed in a field name (e.g. "Combustibile Prev."), so we renamed those fields.

2

Primary keys

We didn't recreate the composite keys: every document gets an automatic `_id` (ObjectId).

3

Foreign keys

There are none in Mongo: we link collections with a lookup only when a query needs it.

Conclusions

Document model

We redid all ten queries: where Postgres uses one optimized statement, in Mongo we spell out the steps: explicit pipelines and small precomputed collections in place of sub-queries.

10

SQL queries
reproduced in Mongo

Graph model

Query 4 is about connected zones, so we rebuilt it in **Neo4j**: the join and group-by turned into a network of congested links we could finally read on a real map.

162

congested links
identical in SQL, Mongo & Neo4j

THANK YOU